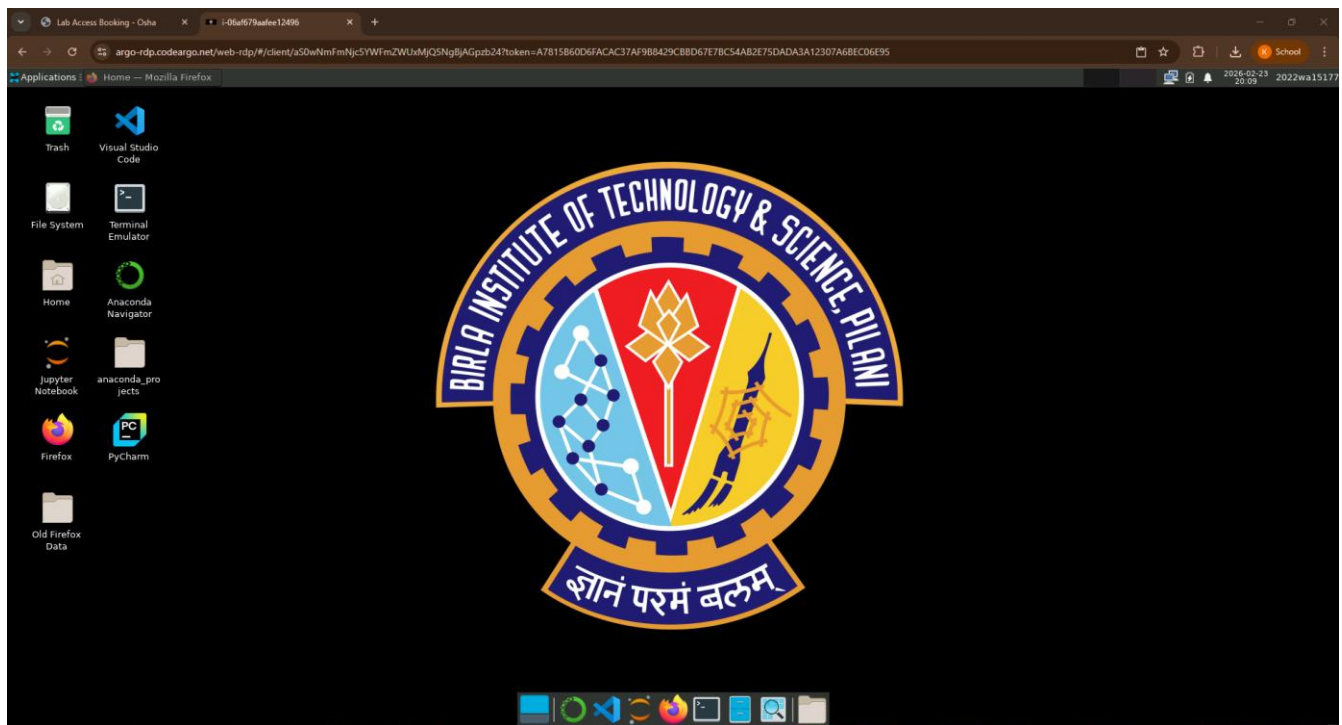


Artificial Intelligence-Lab2

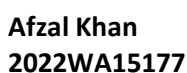
Username Change:

- First, the username is changed to BITS ID - 2022wa15177.

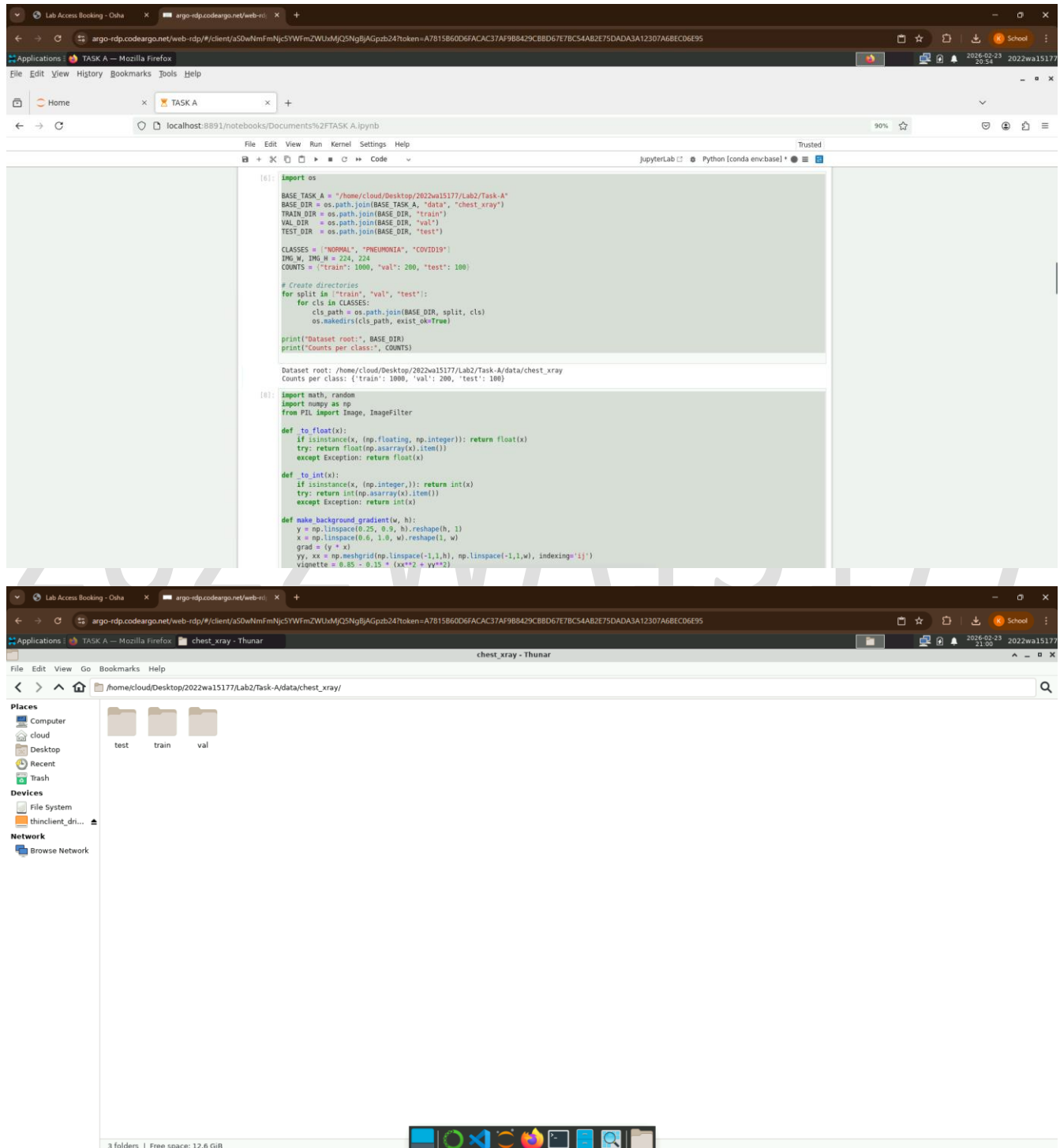


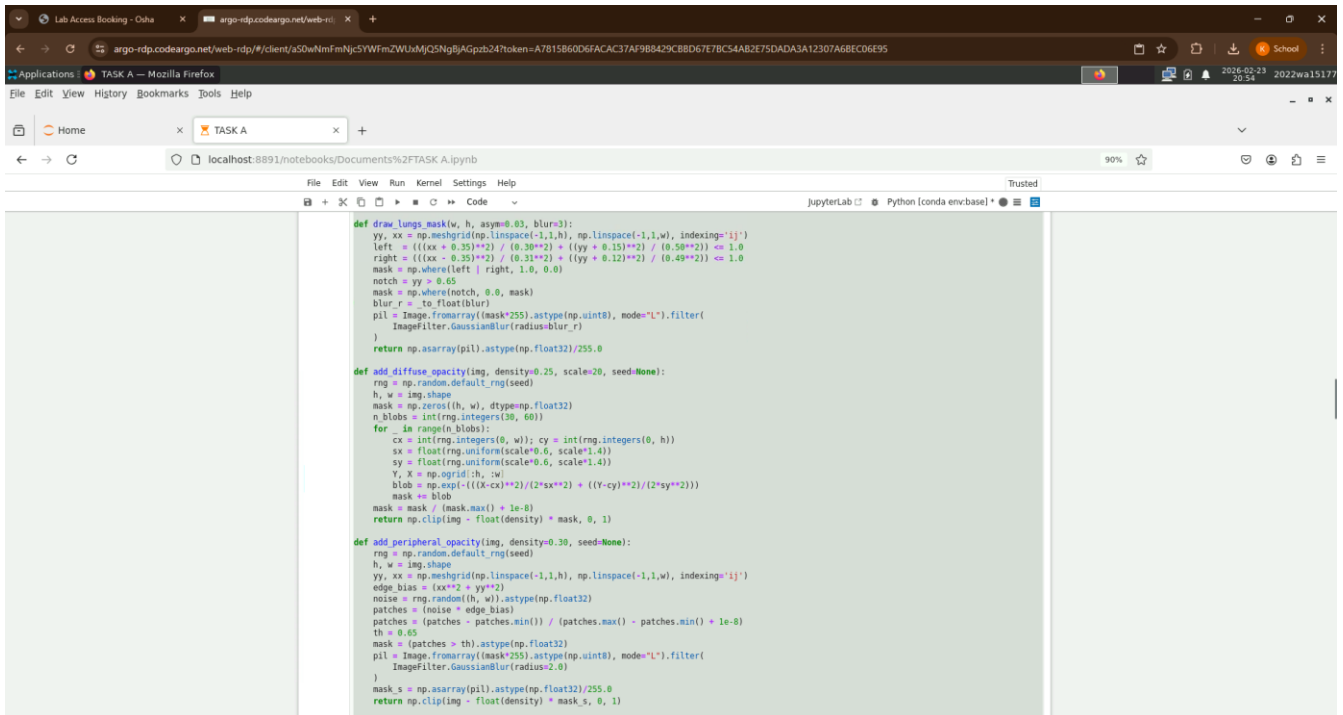
- Username is changed successfully.

- Install the required packages using the command below.



- Create the data folders for training, validation and testing to store data images in them.





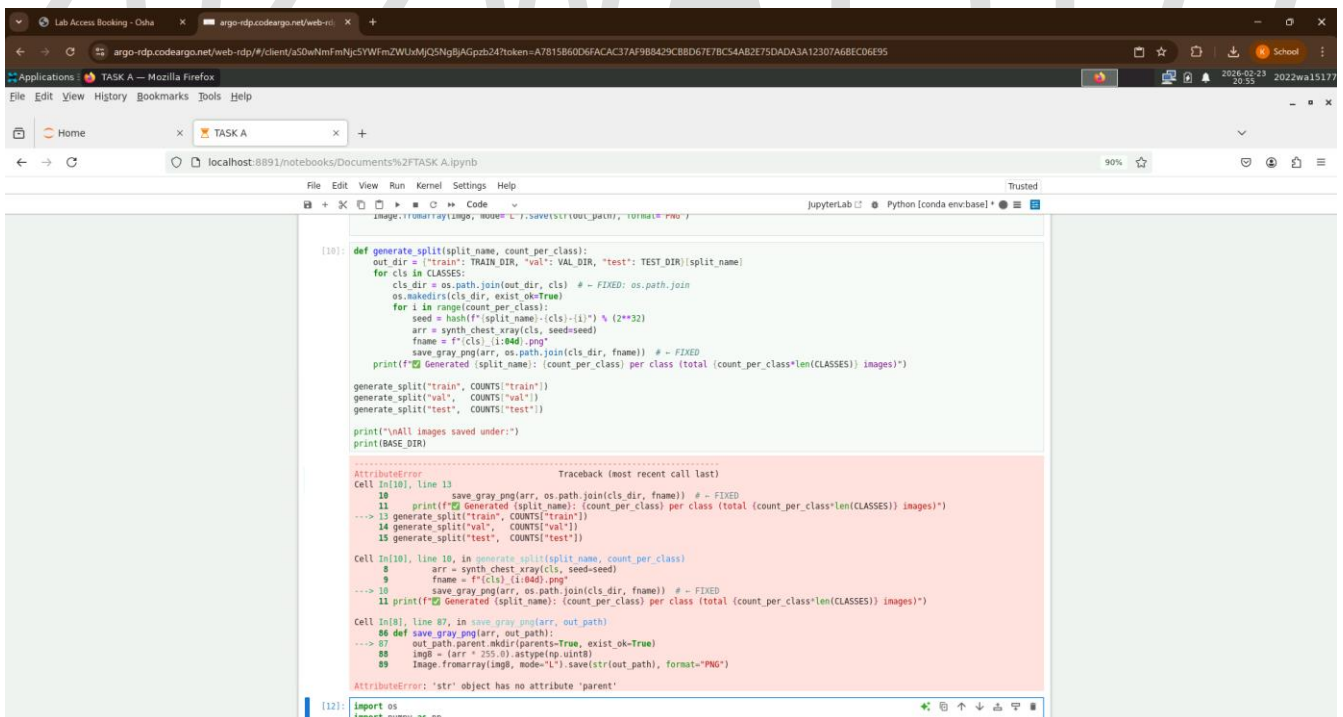
```
def draw_mask(w, h, asym=0.03, blur=3):
    yy, xx = np.meshgrid(np.linspace(-1,1,h), np.linspace(-1,1,w), indexing='ij')
    left = (((xx + 0.35)**2) / (0.30**2)) + (((yy + 0.15)**2) / (0.50**2)) <= 1.0
    right = (((xx - 0.35)**2) / (0.31**2)) + (((yy + 0.12)**2) / (0.49**2)) <= 1.0
    mask = np.where(left | right, 1.0, 0.0)

    notch = yy > 0.65
    mask = np.where(notch, 0.0, mask)
    blur_r = to_float(blur)
    pil = Image.fromarray(mask*255).astype(np.uint8), mode='L').filter(
        ImageFilter.GaussianBlur(radius=blur_r)
    )
    return np.asarray(pil).astype(np.float32)/255.0

def add_diffuse_opacity(img, density=0.25, scale=20, seed=None):
    rng = np.random.default_rng(seed)
    h, w = img.shape
    mask = np.zeros((h, w), dtype=np.float32)
    n_blobs = int(rng.integers(30, 60))
    for _ in range(n_blobs):
        cx = int(rng.integers(0, w)); cy = int(rng.integers(0, h))
        sx = float(rng.uniform(scale*0.6, scale*1.4))
        sy = float(rng.uniform(scale*0.6, scale*1.4))
        Y, X = np.ogrid[:h, :w]
        blob = np.exp(-(((X-cx)**2)/(2*sx**2)) + (((Y-cy)**2)/(2*sy**2)))
        mask += blob
    mask = mask / (mask.max() + 1e-8)
    return np.clip(img - float(density) * mask, 0, 1)

def add_peripheral_opacity(img, density=0.30, seed=None):
    rng = np.random.default_rng(seed)
    h, w = img.shape
    yy, xx = np.meshgrid(np.linspace(-1,1,h), np.linspace(-1,1,w), indexing='ij')
    edge_bias = (xx**2 + yy**2)
    noise = rng.random((h, w)).astype(np.float32)
    patches = (noise + edge_bias)
    patches = (patches - patches.min()) / (patches.max() - patches.min() + 1e-8)
    th = 0.45
    mask = (patches > th).astype(np.float32)
    pil = Image.fromarray(mask*255).astype(np.uint8), mode='L').filter(
        ImageFilter.GaussianBlur(radius=2.0)
    )
    mask_s = np.asarray(pil).astype(np.float32)/255.0
    return np.clip(img - float(density) * mask_s, 0, 1)
```

- Generate the required images for each class (covid, pneumonia, normal).



```
[10]: def generate_split(split_name, count_per_class):
    out_dir = f'train': TRAIN_DIR, 'val': VAL_DIR, 'test': TEST_DIR}[split_name]
    for cls in CLASSES:
        cls_dir = os.path.join(out_dir, cls) # - FIXED: os.path.join
        os.makedirs(cls_dir, exist_ok=True)
        for i in range(count_per_class):
            seed = hash(f'{split_name}-{cls}-{i}') % (2**32)
            arr = synth_chest_xray(cls, seed=seed)
            fname = f'{cls}-{i:04d}.png'
            save_gray_png(arr, os.path.join(cls_dir, fname)) # - FIXED
        print(f'Generated {split_name}: {count_per_class} per class (total {count_per_class*len(CLASSES)} images)')

generate_split('train', COUNTS['train'])
generate_split('val', COUNTS['val'])
generate_split('test', COUNTS['test'])

print('\nAll images saved under:')
print(BASE_DIR)

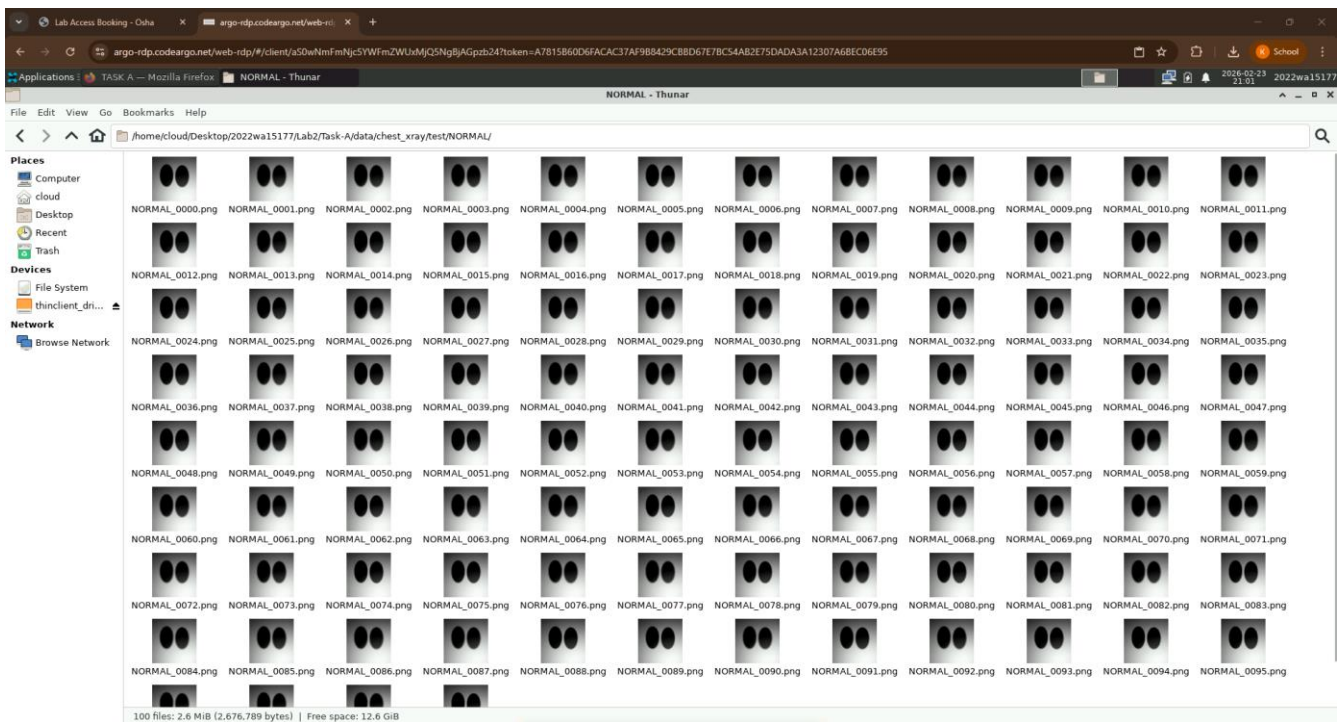
AttributeError                                Traceback (most recent call last)
Cell In[10], line 13
     10         save_gray_png(arr, os.path.join(cls_dir, fname)) # - FIXED
     11     print(f'Generated {split_name}: {count_per_class} per class (total {count_per_class*len(CLASSES)} images)')
--> 13 generate_split('train', COUNTS['train'])
     14 generate_split('val', COUNTS['val'])
     15 generate_split('test', COUNTS['test'])

Cell In[10], line 10, in generate_split(split_name, count_per_class)
     8     arr = synth_chest_xray(cls, seed=seed)
     9     fname = f'{cls}-{i:04d}.png'
--> 10     save_gray_png(arr, os.path.join(cls_dir, fname)) # - FIXED
     11     print(f'Generated {split_name}: {count_per_class} per class (total {count_per_class*len(CLASSES)} images)')

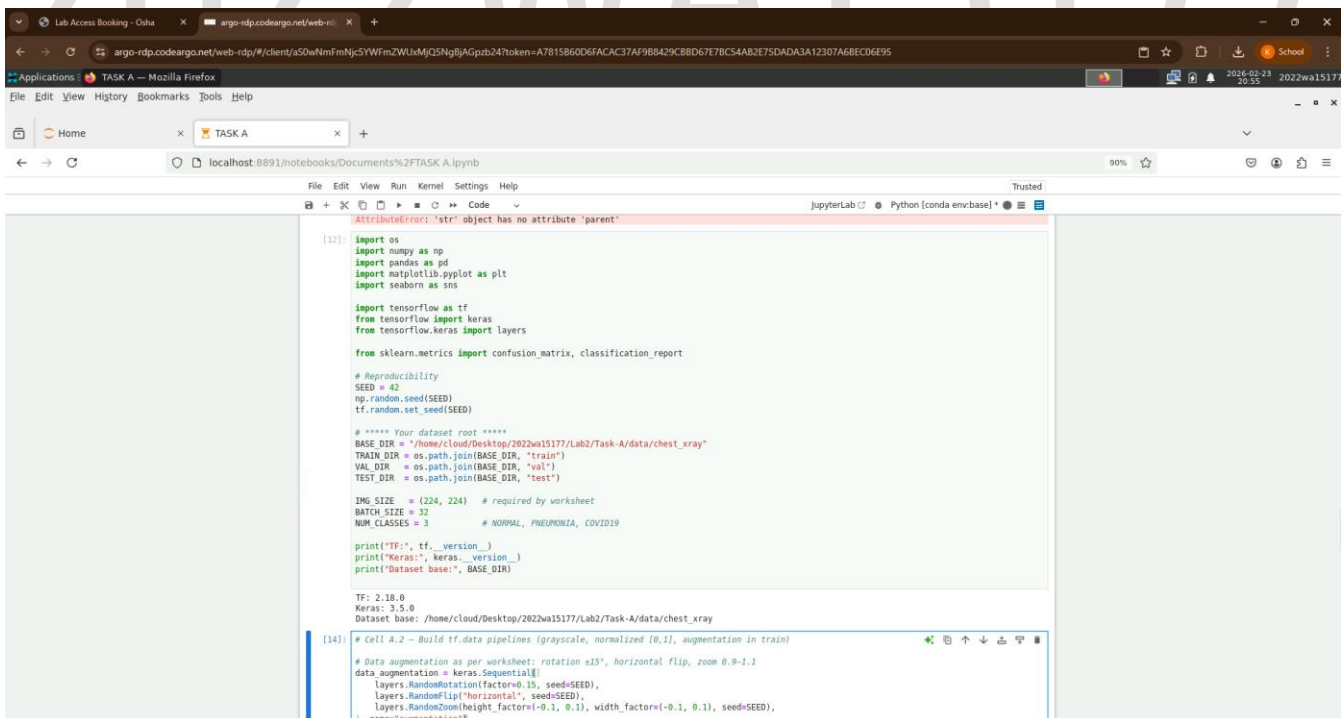
Cell In[8], line 87, in save_gray_png(arr, out_path)
     86 def save_gray_png(arr, out_path):
--> 87     out_path.parent.mkdir(parents=True, exist_ok=True)
     88     img8 = (arr + 255.0).astype(np.uint8)
     89     Image.fromarray(img8, mode='L').save(str(out_path), format='PNG')

AttributeError: 'str' object has no attribute 'parent'

[12]: import os
import numpy as np
```



- Set dataset paths & constants



- Dataset Loading & Augmentation


```
[14]: # Cell A.2 - Build tf.data pipelines (grayscale, normalized [0,1], augmentation in train)

# Data augmentation as per worksheet: rotation ±15°, horizontal flip, zoom 0.9-1.1
data_augmentation = keras.Sequential([
    layers.RandomRotation(factor=0.15, seed=SEED),
    layers.RandomFlip("horizontal", seed=SEED),
    layers.RandomZoom(height_factor=(0.9, 1.1), width_factor=(0.9, 1.1), seed=SEED),
], name="augmentation")

def make_dataset(path, shuffle=True, augment=False):
    ds = keras.utils.image_dataset_from_directory(
        path,
        labels="inferred",
        label_mode="categorical", # one-hot labels
        color_mode="grayscale", # REQUIRED: grayscale
        batch_size=BATCH_SIZE,
        image_size=IMG_SIZE,
        shuffle=shuffle,
        seed=SEED
    )
    # Normalize to [0,1]
    ds = ds.map(lambda x, y: (tf.cast(x, tf.float32)/255.0, y),
                num_parallel_calls=tf.data.AUTOTUNE)
    if augment:
        ds = ds.map(lambda x, y: (data_augmentation(x, training=True), y),
                        num_parallel_calls=tf.data.AUTOTUNE)
    return ds.prefetch(tf.data.AUTOTUNE)

train_ds = make_dataset(TRAIN_DIR, shuffle=True, augment=True)
val_ds = make_dataset(VAL_DIR, shuffle=False, augment=False)
test_ds = make_dataset(TEST_DIR, shuffle=False, augment=False)

class_names = train_ds.class_names
print("Class order:", class_names)

Found 3000 files belonging to 3 classes.
Found 600 files belonging to 3 classes.
Found 300 files belonging to 3 classes.

AttributeError                                Traceback (most recent call last)
Cell [14], line 33
     30 val_ds = make_dataset(VAL_DIR, shuffle=False, augment=False)
     31 test_ds = make_dataset(TEST_DIR, shuffle=False, augment=False)
--> 33 class_names = train_ds.class_names
```

```
AttributeError: 'PrefetchDataset' object has no attribute 'class_names'

[16]: # Patch after A.2 - Infer class names from directory names (sorted for stable order)
import os

class_names = sorted(
    [d for d in os.listdir(TRAIN_DIR) if os.path.isdir(os.path.join(TRAIN_DIR, d))]
)
print("Class order (from folders):", class_names)

Class order (from folders): ['COVID19', 'NORMAL', 'PNEUMONIA']

[18]: # Cell A.3 - Define the CNN (Conv-Pool x3, Dense 128, Dropout 0.5, Softmax 3)
inputs = layers.Input(shape=(IMG_SIZE, 1))
x = layers.Conv2D(32, (3,3), activation="relu", padding="same")(inputs)
x = layers.MaxPooling2D((2,2))(x) # 224 -> 112
x = layers.Conv2D(64, (3,3), activation="relu", padding="same")(x)
x = layers.MaxPooling2D((2,2))(x) # 112 -> 56
x = layers.Conv2D(128, (3,3), activation="relu", padding="same")(x)
x = layers.MaxPooling2D((2,2))(x) # 56 -> 28
x = layers.Flatten()(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = keras.Model(inputs, outputs, name="ChestXrayCNN")
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

Model: "ChestXrayCNN"

```

Layer (type)	Output Shape	Param #
Input Layer 1 (InputLayer)	(None, 224, 224, 1)	0

CNN Model Building

```

x = layers.Flatten()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)

model = keras.Model(inputs, outputs, name='ChestXrayCNN')
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy'])

model.summary()

```

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 1)	0
conv2d (Conv2D)	(None, 224, 224, 32)	320
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten (Flatten)	(None, 100352)	0
dense_1 (Dense)	(None, 128)	12,845,184
dropout (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 3)	387

Total params: 12,938,243 (49.36 MB)
 Trainable params: 12,938,243 (49.36 MB)
 Non-trainable params: 0 (0.00 B)

Model Training

```

[*]: # Cell A.4 - Train the model (epochs=50, early stopping on val_loss)

early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    callbacks=[early_stop],
    verbose=1
)

# Plot training curves
fig, axes = plt.subplots(1, 2, figsize=(12,4))
axes[0].plot(history.history['accuracy'], label='Train Acc')
axes[0].plot(history.history['val_accuracy'], label='Val Acc')
axes[0].set_title('Accuracy over Epochs'); axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Accuracy')
axes[0].grid(True); axes[0].legend()

axes[1].plot(history.history['loss'], label='Train Loss')
axes[1].plot(history.history['val_loss'], label='Val Loss')
axes[1].set_title('Loss over Epochs'); axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Loss')
axes[1].grid(True); axes[1].legend()

plt.tight_layout(); plt.show()

```

```

Epoch 1/50
94/94 — 162s 2s/step - accuracy: 0.5707 - loss: 1.0818 - val_accuracy: 1.0000 - val_loss: 0.0709
Epoch 2/50
94/94 — 152s 2s/step - accuracy: 0.9402 - loss: 0.1732 - val_accuracy: 1.0000 - val_loss: 0.0183
Epoch 3/50
94/94 — 156s 2s/step - accuracy: 0.9679 - loss: 0.0900 - val_accuracy: 1.0000 - val_loss: 0.0043
Epoch 4/50
11/94 — 2:11 2s/step - accuracy: 0.9693 - loss: 0.0876

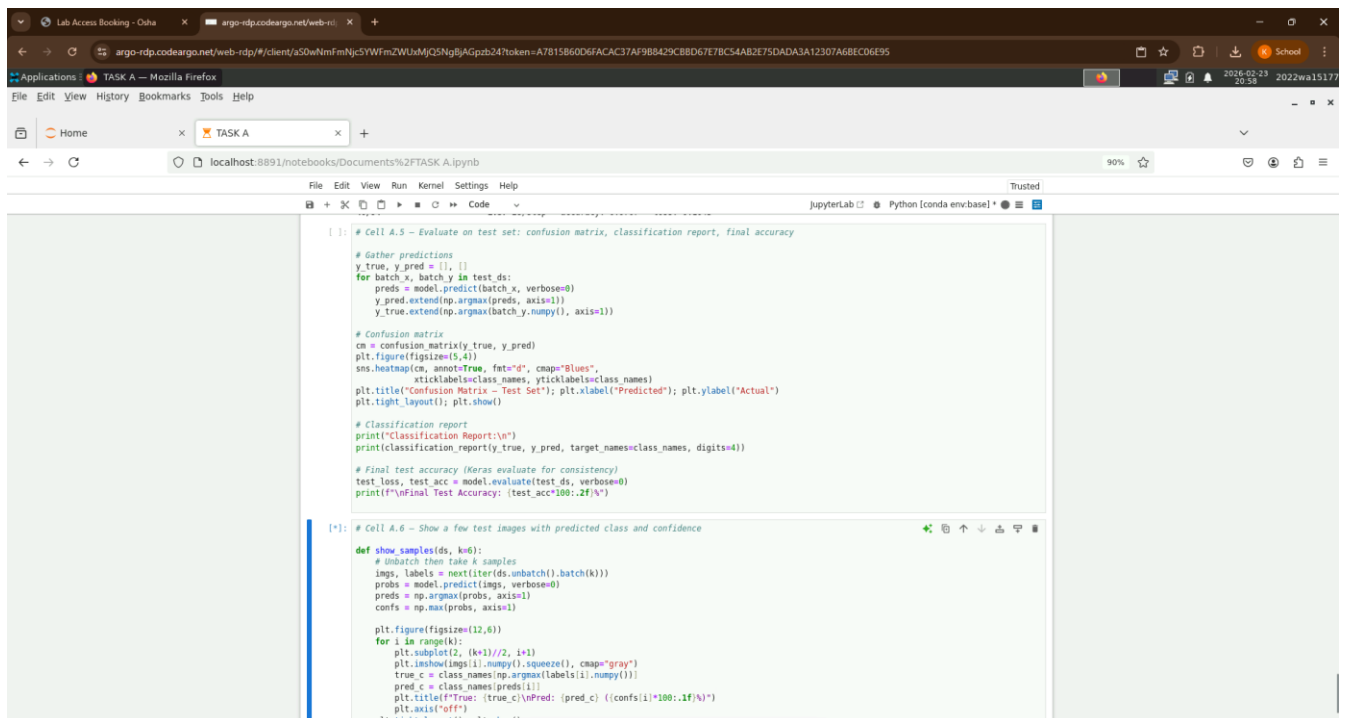
```

```

[*]: # Cell A.5 - Evaluate on test set: confusion matrix, classification report, final accuracy

# Gather predictions

```



```
[ ]: # Cell A.5 - Evaluate on test set: confusion matrix, classification report, final accuracy

# Gather predictions
y_true, y_pred = [], []
for batch_x, batch_y in test_ds:
    preds = model.predict(batch_x, verbose=0)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(np.argmax(batch_y.numpy(), axis=1))

# Confusion matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.title("Confusion Matrix - Test Set"); plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout(); plt.show()

# Classification report
print("Classification Report:\n")
print(classification_report(y_true, y_pred, target_names=class_names, digits=4))

# Final test accuracy (Keras evaluate for consistency)
test_loss, test_acc = model.evaluate(test_ds, verbose=0)
print(f"Final Test Accuracy: {test_acc*100:.2f}%")

[ ]: # Cell A.6 - Show a few test images with predicted class and confidence

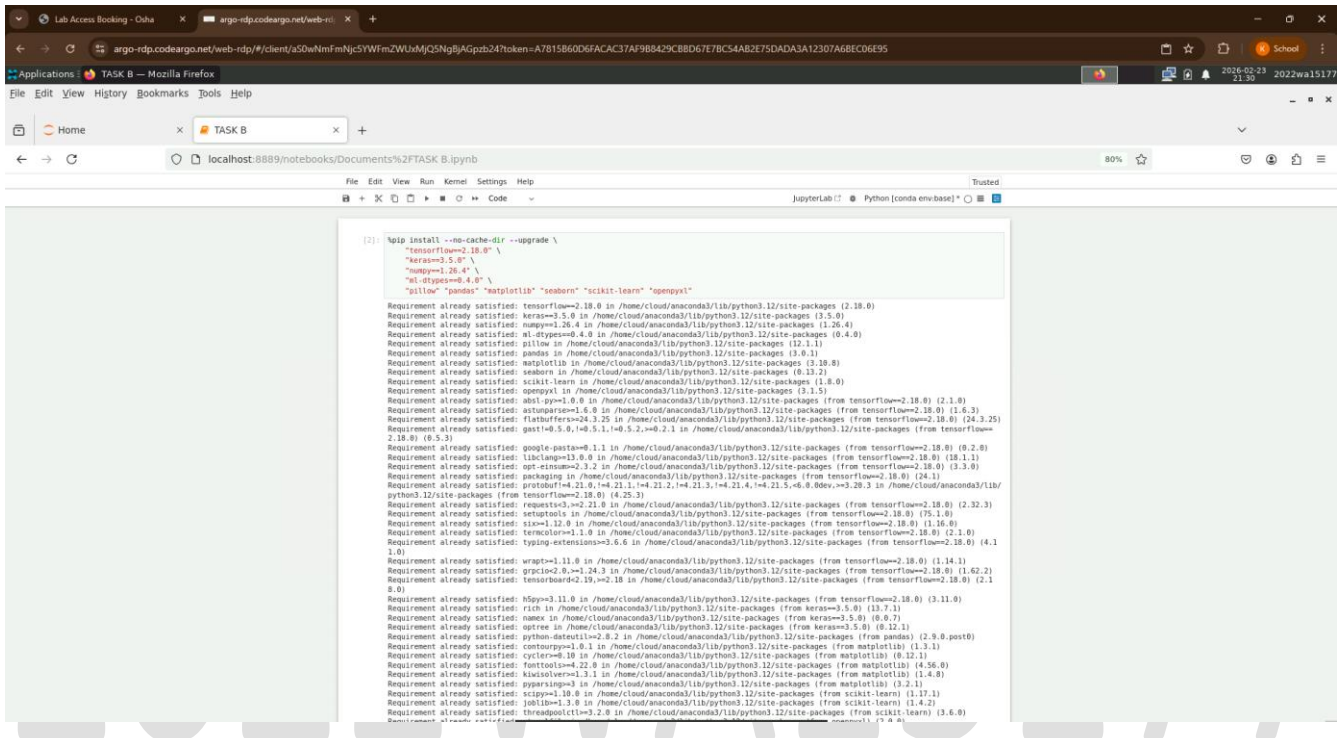
def show_samples(ds, k=6):
    # Unbatch then take k samples
    imgs, labels = next(iter(ds.unbatch().batch(k)))
    probs = model.predict(imgs, verbose=0)
    preds = np.argmax(probs, axis=1)
    confs = np.max(probs, axis=1)

    plt.figure(figsize=(12,6))
    for i in range(k):
        plt.subplot(2, (k+1)//2, i+1)
        plt.imshow(imgs[i].numpy().squeeze(), cmap="gray")
        true_c = class_names[np.argmax(labels[i].numpy())]
        pred_c = class_names[preds[i]]
        plt.title(f"True: {true_c}\nPred: {pred_c} ({confs[i]*100:.2f}%)")
        plt.axis('off')
    plt.tight_layout(); plt.show()
```

2022WA15177

Task – B: Stock Price Prediction using RNN

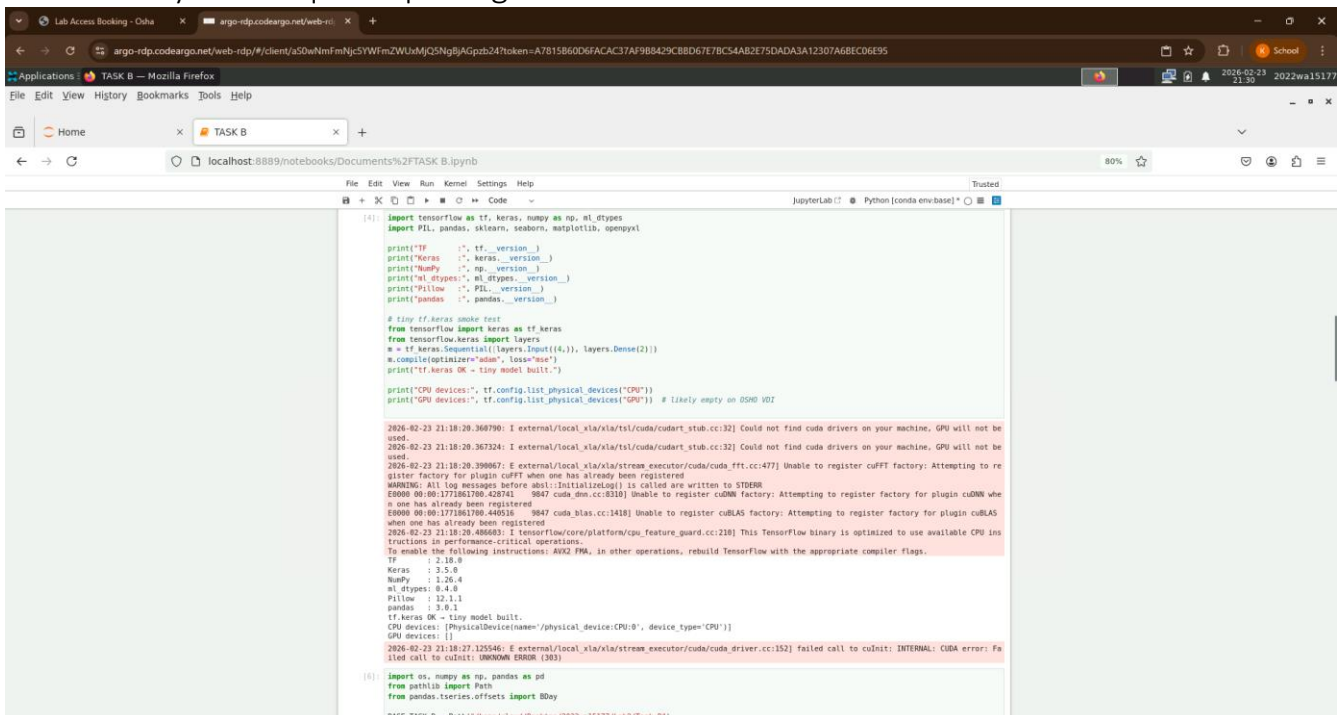
- Install the required packages using the command below.



```
[2]: pip install --no-cache-dir --upgrade \
    "tensorflow==2.10.0" \
    "keras==3.0.0" \
    "numpy==1.26.4" \
    "pillow==10.4.0" \
    "matplotlib==3.8.0" \
    "pandas==2.1.4" \
    "seaborn==0.13.2" \
    "scikit-learn==1.4.2" \
    "openpyxl==3.1.2"

Requirement already satisfied: tensorflow==2.10.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (2.10.0)
Requirement already satisfied: keras==3.0.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (3.0.0)
Requirement already satisfied: numpy==1.26.4 in /home/cloud/anaconda3/lib/python3.12/site-packages (1.26.4)
Requirement already satisfied: pillow==10.4.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (10.4.0)
Requirement already satisfied: matplotlib==3.8.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (3.8.0)
Requirement already satisfied: pandas==2.1.4 in /home/cloud/anaconda3/lib/python3.12/site-packages (2.1.4)
Requirement already satisfied: seaborn==0.13.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (0.13.2)
Requirement already satisfied: openpyxl==3.1.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (3.1.2)
Requirement already satisfied: scikit-learn==1.4.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (1.4.2)
Requirement already satisfied: openpyxl==3.1.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (3.1.2)
Requirement already satisfied: astunparse==1.6.3 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (1.6.3)
Requirement already satisfied: flatbuffers==24.3.25 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (24.3.25)
Requirement already satisfied: gast==0.5.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (0.5.0)
Requirement already satisfied: google-pasta==0.2.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (0.2.0)
Requirement already satisfied: libclang==13.0.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (13.0.0)
Requirement already satisfied: opt-einsum==3.3.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (3.3.0)
Requirement already satisfied: packaging==24.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (24.1)
Requirement already satisfied: protobuf==4.25.3 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (4.25.3)
Requirement already satisfied: requests==2.32.3 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (2.32.3)
Requirement already satisfied: setuptools==75.1.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (75.1.0)
Requirement already satisfied: six==1.16.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (1.16.0)
Requirement already satisfied: tensorboard==2.10.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (2.10.0)
Requirement already satisfied: typing-extensions==4.12.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (4.12.2)
Requirement already satisfied: wrapt==1.15.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (1.15.0)
Requirement already satisfied: grpcio==1.64.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (1.64.0)
Requirement already satisfied: tensorboard==2.10.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (2.10.0)
Requirement already satisfied: h5py==3.11.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (3.11.0)
Requirement already satisfied: rich in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (13.7.1)
Requirement already satisfied: name in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (0.0.7)
Requirement already satisfied: optree in /home/cloud/anaconda3/lib/python3.12/site-packages (from tensorflow==2.10.0) (0.12.1)
Requirement already satisfied: python-dateutil==2.9.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from pandas==2.1.4) (2.9.0)
Requirement already satisfied: contourpy==1.1.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib==3.8.0) (1.1.1)
Requirement already satisfied: cycler==0.12.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib==3.8.0) (0.12.1)
Requirement already satisfied: fonttools==4.56.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib==3.8.0) (4.56.0)
Requirement already satisfied: kiwisolver==1.4.7 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib==3.8.0) (1.4.7)
Requirement already satisfied: pyparsing==3.1.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib==3.8.0) (3.1.2)
Requirement already satisfied: scipy==1.13.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn==1.4.2) (1.13.1)
Requirement already satisfied: joblib==1.4.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn==1.4.2) (1.4.2)
Requirement already satisfied: threadpoolctl==3.3.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn==1.4.2) (3.3.0)
```

- Verify if all required packages are installed as below.



```
[4]: import tensorflow as tf, keras, numpy as np, matplotlib
import PIL, pandas, sklearn, seaborn, matplotlib, openpyxl

print("tf", tf.__version__)
print("keras", keras.__version__)
print("numpy", np.__version__)
print("pillow", PIL.__version__)
print("matplotlib", matplotlib.__version__)
print("pandas", pandas.__version__)

# tiny tf.keras model test
from tensorflow.keras import layers
m = tf.keras.Sequential(layers.Dense(4), layers.Dense(2))
m.compile(optimizer="adam", loss="mse")
print("tf.keras OK - tiny model built.")

print("CPU devices:", tf.config.list_physical_devices("CPU"))
print("GPU devices:", tf.config.list_physical_devices("GPU")) # Likely empty on OSHO VDI

2026-02-23 21:18:20.360790: I external/local_xla/xla/ts/cuda/cudart_stub.cc:322 Could not find cuda drivers on your machine, GPU will not be used.
2026-02-23 21:18:20.367224: I external/local_xla/xla/ts/cuda/cudart_stub.cc:322 Could not find cuda drivers on your machine, GPU will not be used.
2026-02-23 21:18:20.390687: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1771861700.420741 8047 cuda_device.cc:8210] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 00:00:1771861700.440518 8047 cuda_device.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered
2026-02-23 21:18:20.486683: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2, FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
TF
Keras : 3.0.0
numpy : 1.26.4
pillow : 10.4.0
matplotlib : 3.8.0
pandas : 2.1.4
tf.keras OK - tiny model built.
CPU devices: [PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU')]
GPU devices: []

2026-02-23 21:18:27.125546: E external/local_xla/xla/stream_executor/cuda/cuda_driver.cc:352] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (383)

[5]: import os, numpy as np, pandas as pd
from pathlib import Path
from pandas.tseries.offsets import BDay

BASE_TASK_B = Path("/home/cloud/Desktop/2022wa15177/lab2/Task-B")
```

- Generate Stock Dataset

```

import os, numpy as np, pandas as pd
from pathlib import Path
from pandas.tseries.offsets import BDay

BASE_TASK_B = Path("/home/cloud/Desktop/2022wa15177/Lab2/Task-B")
STOCK_DIR = BASE_TASK_B / "data" / "stocks"
STOCK_DIR.mkdir(parents=True, exist_ok=True)

TICKER = "AAPL"
YEARS = 5
TRADING_DAYS_PER_YEAR = 252
N = YEARS * TRADING_DAYS_PER_YEAR

today = pd.Timestamp.today().normalize().tz_localize(None)

if today.weekday() == 5:
    shift = today.weekday() - 4
    end_bd = today - BDay(shift)
else:
    end_bd = today

while len(pd.date_range(end=end_bd, periods=1, freq="B")) == 0:
    end_bd = end_bd - BDay(1)

dates = pd.date_range(end=end_bd, periods=N, freq="B")
assert len(dates) == N, f"Expected {N} business days, got {len(dates)}"
print(f"Business-day window: {dates[0].date()} - {dates[-1].date()} (rows: {len(dates)})")

np.random.seed(42)
start_price = 150.0
mu = 0.0002 # drift
sigma = 0.02 # daily volatility

eps = np.random.normal(0, 1, size=N)
close = [start_price]
for i in range(1, N):
    close.append(close[-1] * np.exp((mu - 0.5*sigma**2) * 1 + sigma*eps[i]))
close = np.array(close)

open_ = np.concatenate([close[0], close[1:] * (1.0 + np.random.normal(0, 0.002, size=N))])

span = np.abs(close - open_)
high = np.maximum(open_, close) + np.maximum(0, 0.5*span) * (0.2 + np.random.rand(N))
low = np.minimum(open_, close) - np.maximum(0, 0.5*span) * (0.2 + np.random.rand(N))
high = np.maximum(high, np.maximum(open_, close))
low = np.minimum(low, np.minimum(open_, close))

base_vol = 3_000_000

vol = (np.abs(eps) * 1_000_000 + base_vol).astype(int)

df = pd.DataFrame({
    "Date": dates,
    "Open": open_,
    "High": high,
    "Low": low,
    "Close": close,
    "Volume": vol
})

xlsx_path = STOCK_DIR / f"{TICKER}_Sy.xlsx"
csv_path = STOCK_DIR / f"{TICKER}_Sy.csv"

with pd.ExcelWriter(xlsx_path, engine="openpyxl") as writer:
    df.to_excel(writer, index=False, sheet_name="OHLCV")
df.to_csv(csv_path, index=False)

print(f"Saved Excel: {xlsx_path}")
print(f"Saved CSV: {csv_path}")
print(df.head(), "\n...", df.tail())

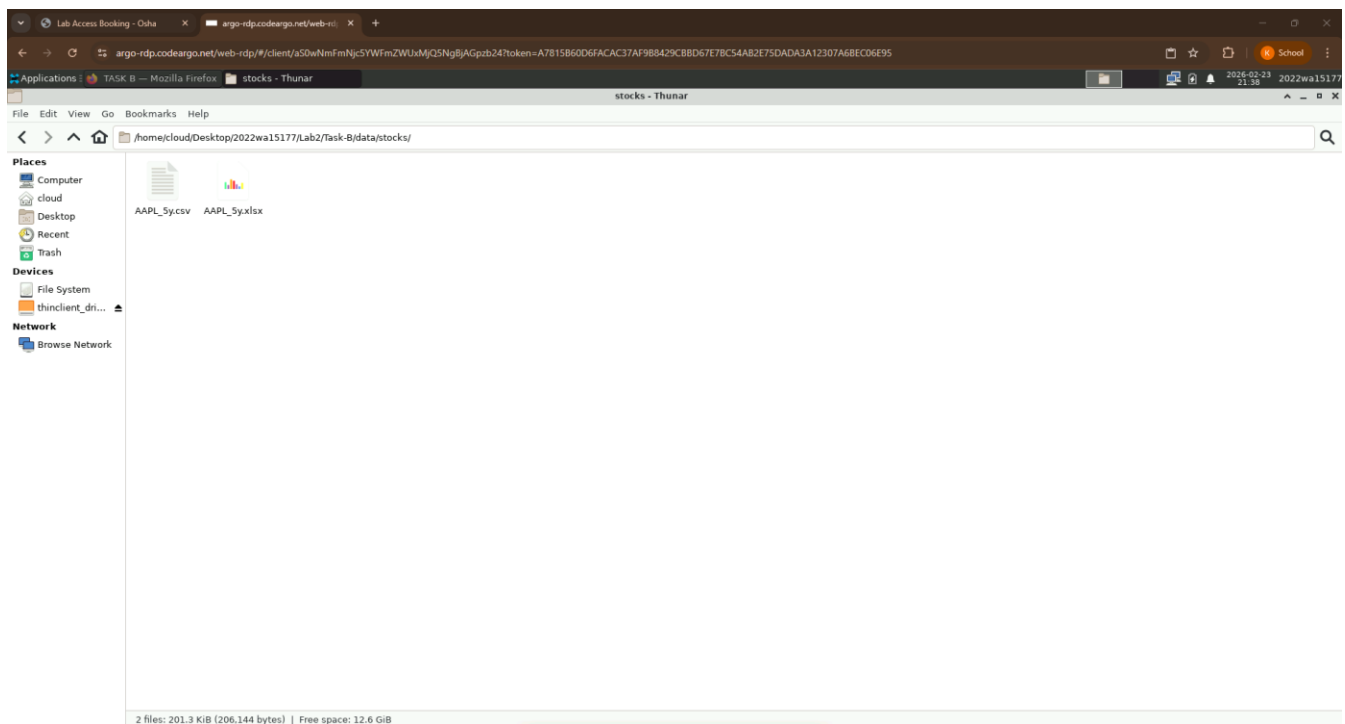
Business-day window: 2021-04-27 - 2026-02-23 (rows: 1260)
Saved Excel: /home/cloud/Desktop/2022wa15177/Lab2/Task-B/data/stocks/AAPL_Sy.xlsx
Saved CSV: /home/cloud/Desktop/2022wa15177/Lab2/Task-B/data/stocks/AAPL_Sy.csv

Date      Open      High      Low      Close  Volume
0 2021-04-27 150.167488 150.262055 149.939726 150.009080 3490714
1 2021-04-28 150.022802 150.176444 149.429337 149.585780 3136264
2 2021-04-29 149.746061 151.088439 148.676594 151.536085 3647608
3 2021-04-30 151.257854 158.851990 150.424721 156.222985 4523829
4 2021-05-03 156.275901 156.619339 155.150435 155.493992 3234153
...
Date      Open      High      Low      Close  Volume
1255 2026-02-17 387.426318 388.660568 375.533496 377.818922 4269641
1256 2026-02-18 377.982943 396.478773 375.499592 390.773750 4672572
1257 2026-02-19 389.030261 394.936226 387.774110 394.862343 2425913
1258 2026-02-20 393.081232 395.524320 387.763150 388.544960 3765011
1259 2026-02-23 387.883341 388.197346 387.844056 388.111826 3055769

import pandas as pd
import numpy as np

df = pd.read_excel(xlsx_path, engine="openpyxl")
df["Date"] = pd.to_datetime(df["Date"], errors="coerce").dt.tz_localize(None)
df = df.sort_values("Date").reset_index(drop=True)

```



- Load & Clean Data
- Apply MinMax scaling and generates 60-day input sequences for LSTM prediction.

```

import pandas as pd
import numpy as np

df = pd.read_excel(xlsx_path, engine='openpyxl')

df['Date'] = pd.to_datetime(df['Date'], errors='coerce').dt.tz_localize(None)
df = df.sort_values('Date').reset_index(drop=True)
df_ffill = df.ffill().bfill()

features = ["Open", "High", "Low", "Close", "Volume"]
data = df.ffill(features).astype(float).values
dates = df.ffill('Date').values

print('Rows:', len(df_ffill), ' Feature matrix shape:', data.shape)
print('No NaN remaining in features:', np.isnan(data).sum())
print(df_ffill.head(3))

Rows: 1268 | Feature matrix shape: (1268, 5)
No NaN remaining in features: 0
   Date      Open      High      Low      Close  Volume
0 2021-04-27  150.167408  150.242635  149.939726  150.000000  3496734
1 2021-04-28  150.022802  150.176444  149.428537  149.585780  3138264
2 2021-04-29  149.746061  151.088428  148.670594  151.530805  3647688

[18]: import numpy as np
      from sklearn.preprocessing import MinMaxScaler

      scaler = MinMaxScaler(feature_range=(0, 1))
      scaled = scaler.fit_transform(data)

      WINDOW = 60
      X, y = [], []
      for i in range(WINDOW, len(scaled)):
          X.append(scaled[i-WINDOW:i, :])
          y.append(scaled[i, 3])

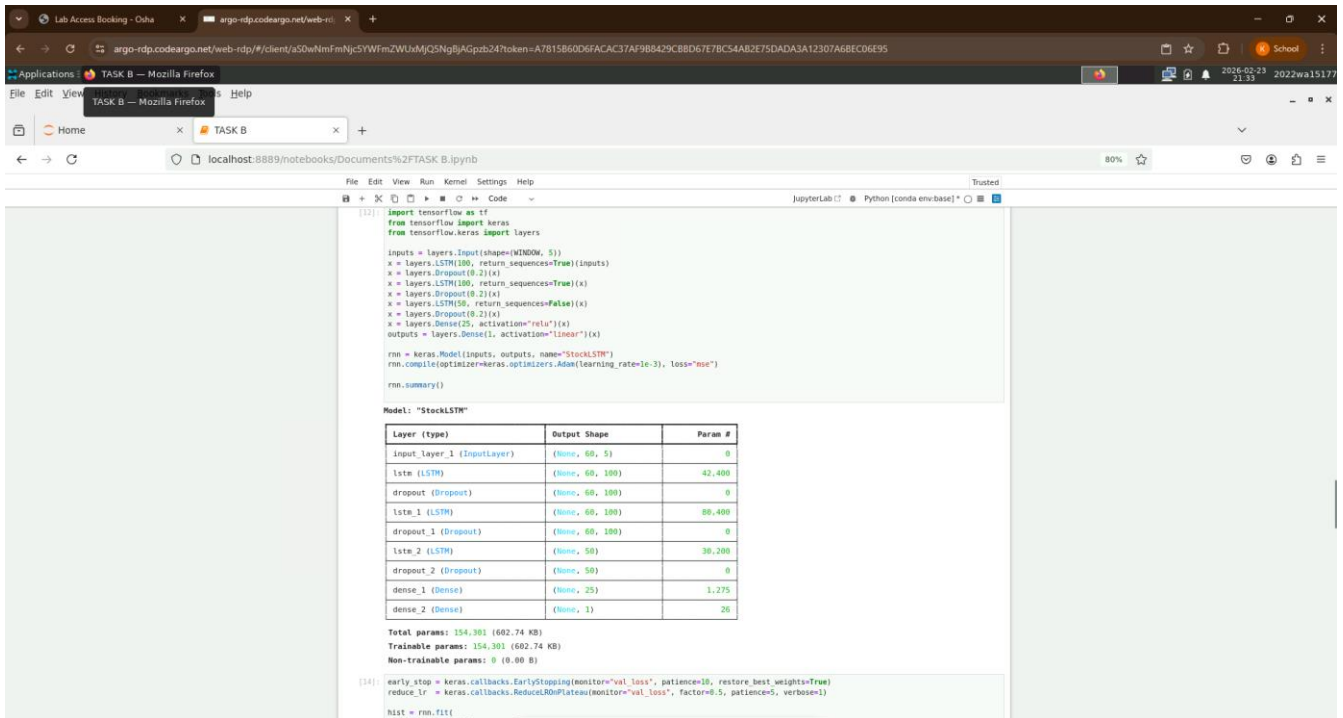
      X, y = np.array(X), np.array(y)
      print('X shape:', X.shape, 'y shape:', y.shape)

      split_idx = int(0.8 * len(X))
      X_train, y_train = X[:split_idx], y[:split_idx]
      X_test, y_test = X[split_idx:], y[split_idx:]
      print('Train:', X_train.shape, 'Test:', X_test.shape)
      X shape: (1200, 60, 5) y shape: (1200,)
      Train: (960, 60, 5) Test: (240, 60, 5)

[12]: import tensorflow as tf
      from tensorflow.keras.layers import LSTM

```

- Build LSTM Model



```
[12]: import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

inputs = layers.Input(shape=(WINDOW, 5))
x = layers.LSTM(100, return_sequences=True)(inputs)
x = layers.Dropout(0.2)(x)
x = layers.LSTM(100, return_sequences=True)(x)
x = layers.Dropout(0.2)(x)
x = layers.LSTM(50, return_sequences=False)(x)
x = layers.Dropout(0.2)(x)
x = layers.Dense(25, activation='relu')(x)
outputs = layers.Dense(1, activation='linear')(x)

m = keras.Model(inputs, outputs, name='StockLSTM')
m.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3), loss='mse')

m.summary()
```

Model: "StockLSTM"

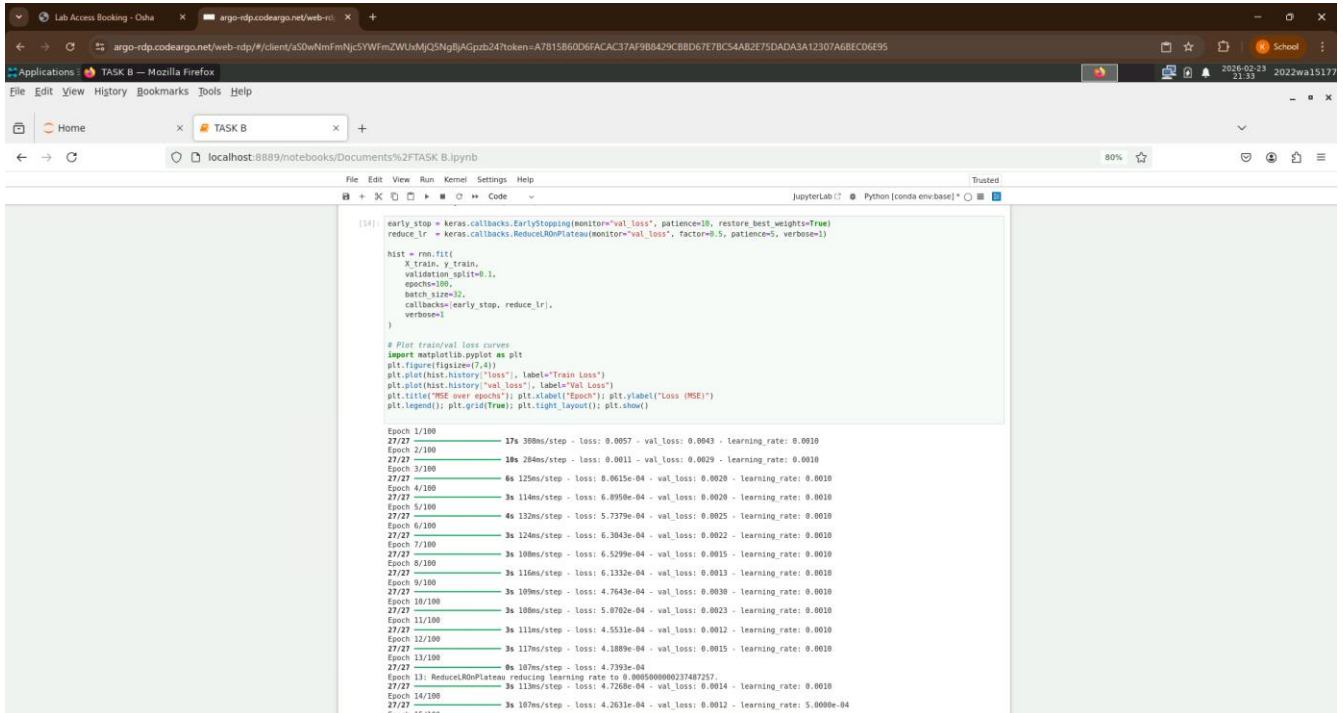
Layer (type)	Output Shape	Param #
Input Layer_1 (InputLayer)	(None, 60, 5)	0
lstm_1 (LSTM)	(None, 60, 100)	42,400
dropout_1 (Dropout)	(None, 60, 100)	0
lstm_2 (LSTM)	(None, 60, 100)	86,400
dropout_2 (Dropout)	(None, 60, 100)	0
lstm_3 (LSTM)	(None, 60)	38,200
dropout_3 (Dropout)	(None, 60)	0
dense_1 (Dense)	(None, 25)	1,275
dense_2 (Dense)	(None, 1)	26

Total params: 154,301 (602.74 KB)
Trainable params: 154,301 (602.74 KB)
Non-trainable params: 0 (0.00 KB)

```
[14]: early_stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
reduce_lr = keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, verbose=1)

hist = m.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    callbacks=[early_stop, reduce_lr],
    verbose=1)
```

- Train LSTM Model

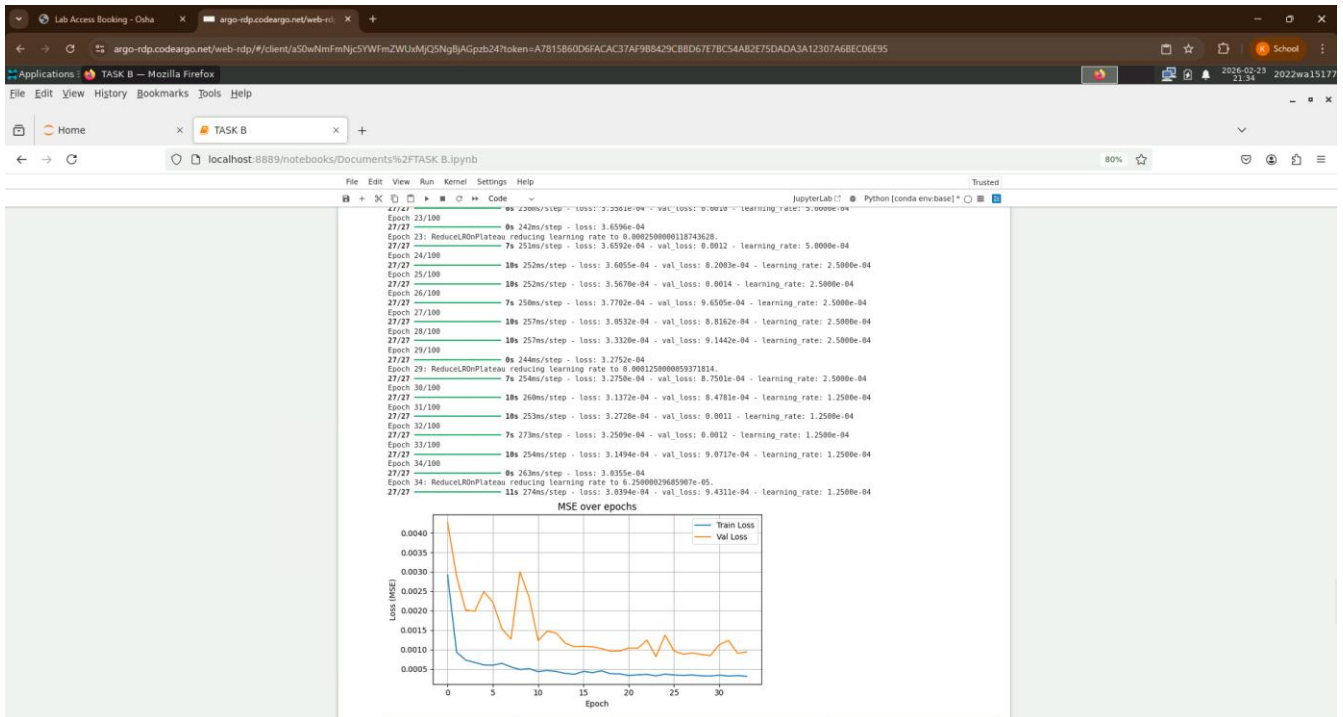


```
[14]: early_stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
reduce_lr = keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, verbose=1)

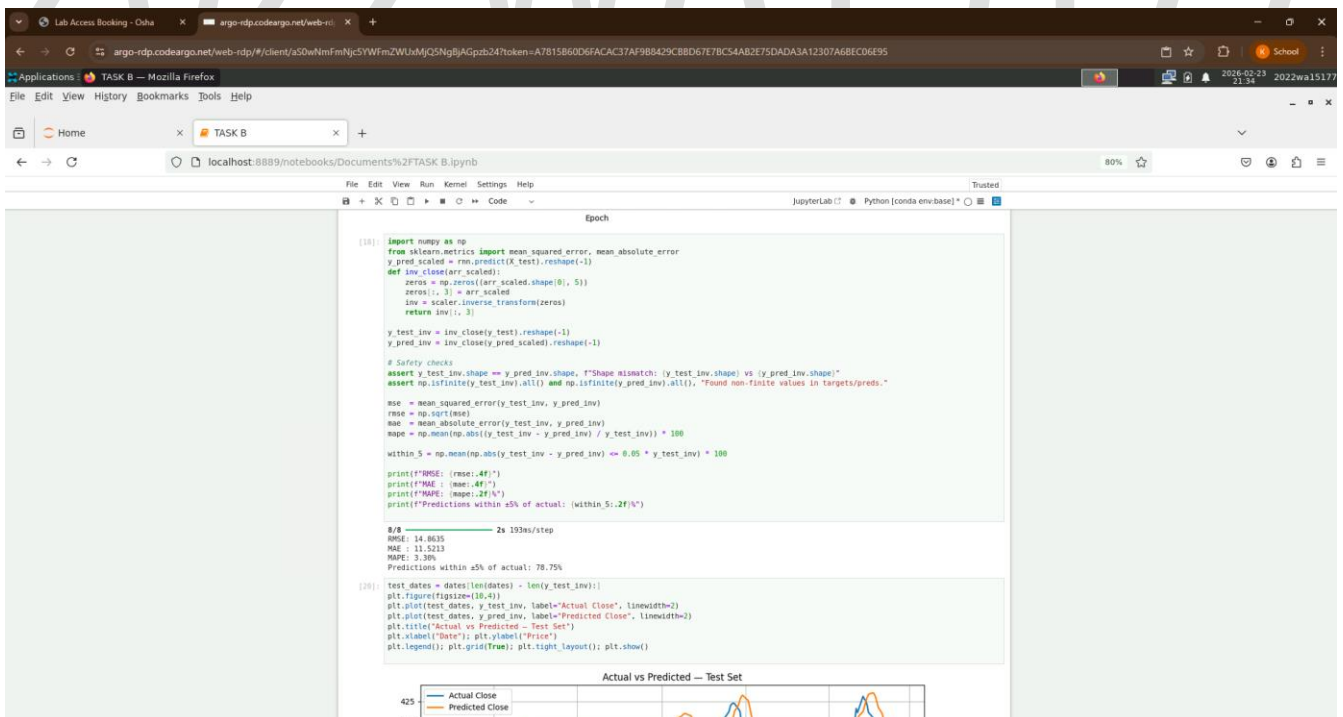
hist = m.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    callbacks=[early_stop, reduce_lr],
    verbose=1)

# Plot train/val loss curves
import matplotlib.pyplot as plt
plt.figure(figsize=(7,4))
plt.plot(hist.history['loss'], label='Train Loss')
plt.plot(hist.history['val_loss'], label='Val Loss')
plt.title("MSE over epochs"); plt.xlabel("Epoch"); plt.ylabel("loss (MSE)")
plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()
```

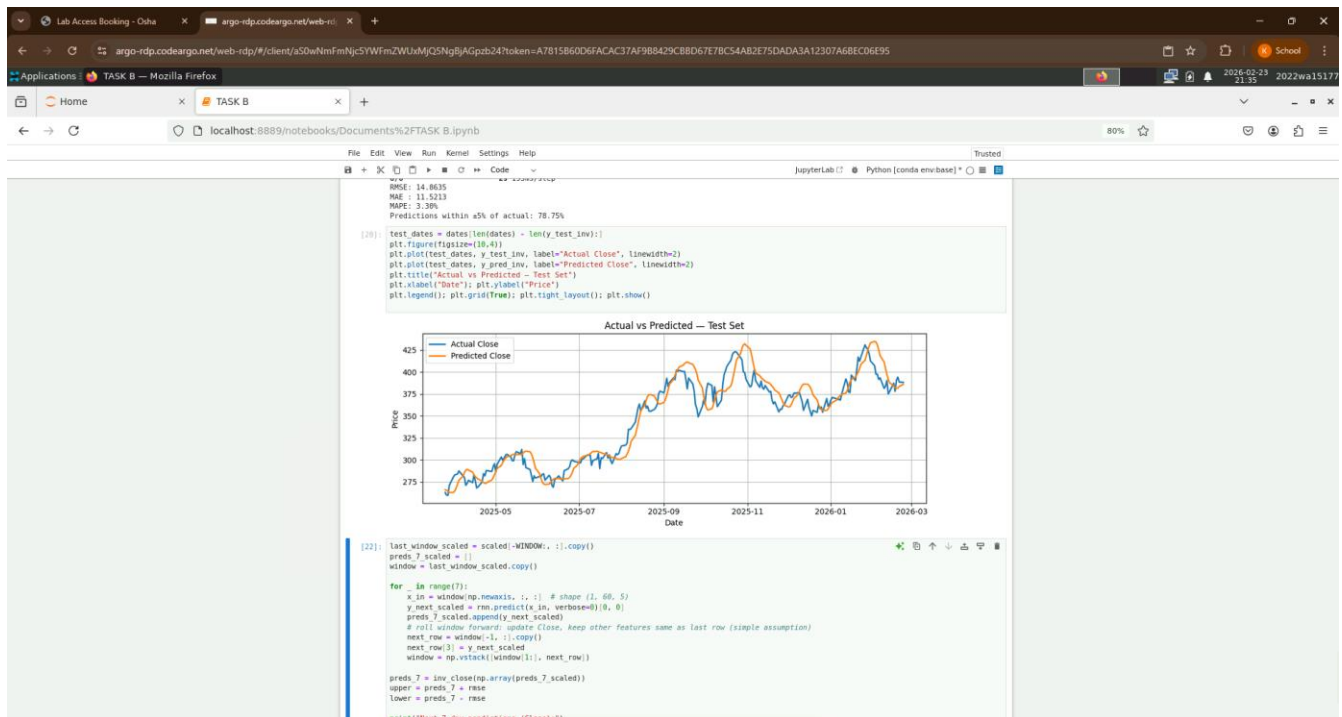
Epoch 1/100
27/27 — 17s 308ms/step - loss: 0.0057 - val_loss: 0.0043 - learning_rate: 0.0010
Epoch 2/100
27/27 — 18s 284ms/step - loss: 0.0011 - val_loss: 0.0029 - learning_rate: 0.0010
Epoch 3/100
27/27 — 6s 125ms/step - loss: 0.0015e-04 - val_loss: 0.0020 - learning_rate: 0.0010
Epoch 4/100
27/27 — 3s 114ms/step - loss: 6.8950e-04 - val_loss: 0.0020 - learning_rate: 0.0010
Epoch 5/100
27/27 — 4s 132ms/step - loss: 5.7379e-04 - val_loss: 0.0025 - learning_rate: 0.0010
Epoch 6/100
27/27 — 3s 124ms/step - loss: 6.3043e-04 - val_loss: 0.0022 - learning_rate: 0.0010
Epoch 7/100
27/27 — 3s 108ms/step - loss: 6.5299e-04 - val_loss: 0.0015 - learning_rate: 0.0010
Epoch 8/100
27/27 — 3s 116ms/step - loss: 6.1332e-04 - val_loss: 0.0013 - learning_rate: 0.0010
Epoch 9/100
27/27 — 3s 109ms/step - loss: 4.7643e-04 - val_loss: 0.0030 - learning_rate: 0.0010
Epoch 10/100
27/27 — 3s 108ms/step - loss: 5.0702e-04 - val_loss: 0.0023 - learning_rate: 0.0010
Epoch 11/100
27/27 — 3s 111ms/step - loss: 4.5531e-04 - val_loss: 0.0012 - learning_rate: 0.0010
Epoch 12/100
27/27 — 3s 117ms/step - loss: 4.1809e-04 - val_loss: 0.0015 - learning_rate: 0.0010
Epoch 13/100
27/27 — 0s 107ms/step - loss: 4.7393e-04
Epoch 13: ReduceLROnPlateau reducing learning rate to 0.0005000000021487237
27/27 — 3s 113ms/step - loss: 4.7268e-04 - val_loss: 0.0014 - learning_rate: 0.0010
Epoch 14/100
27/27 — 3s 107ms/step - loss: 4.2631e-04 - val_loss: 0.0012 - learning_rate: 5.0000e-04
Epoch 15/100



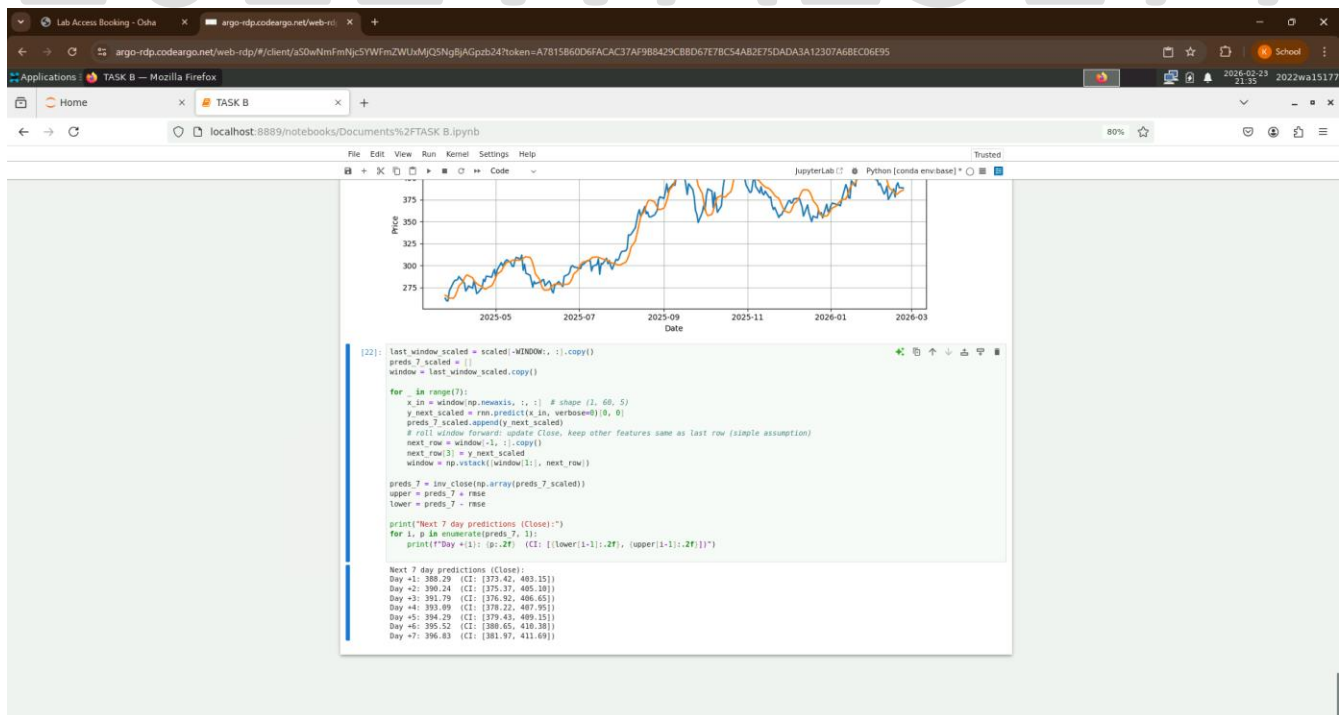
- Compute Evaluation Metrics



- Actual vs Predicted Plot



- Next 7-Day Forecast



Lab Usage Time Snapshot:

Lab Access Booking - Osha

osha-bits.codeargo.net/booking/

School

Osha

KHAN AFZAL AMANULLAH NUZHAT

Course / Experiment:
--- Select Course / Experiment ---

Date:
--- Select Date ---

Slot:
--- Select Slot ---

Book

Course (Virtual Lab)	Machine	Time Used	Time Remaining	State	Control
2551WT-45-Cryptography(Virtual Lab)	i-085e4ac9f619a830	1h 41m	20h 18m	saved	Start
2551WT-45-AI(Virtual Lab)	i-06af679aafef12496	1h 52m	20h 7m	saved	Start

© Code Argo. All rights reserved.